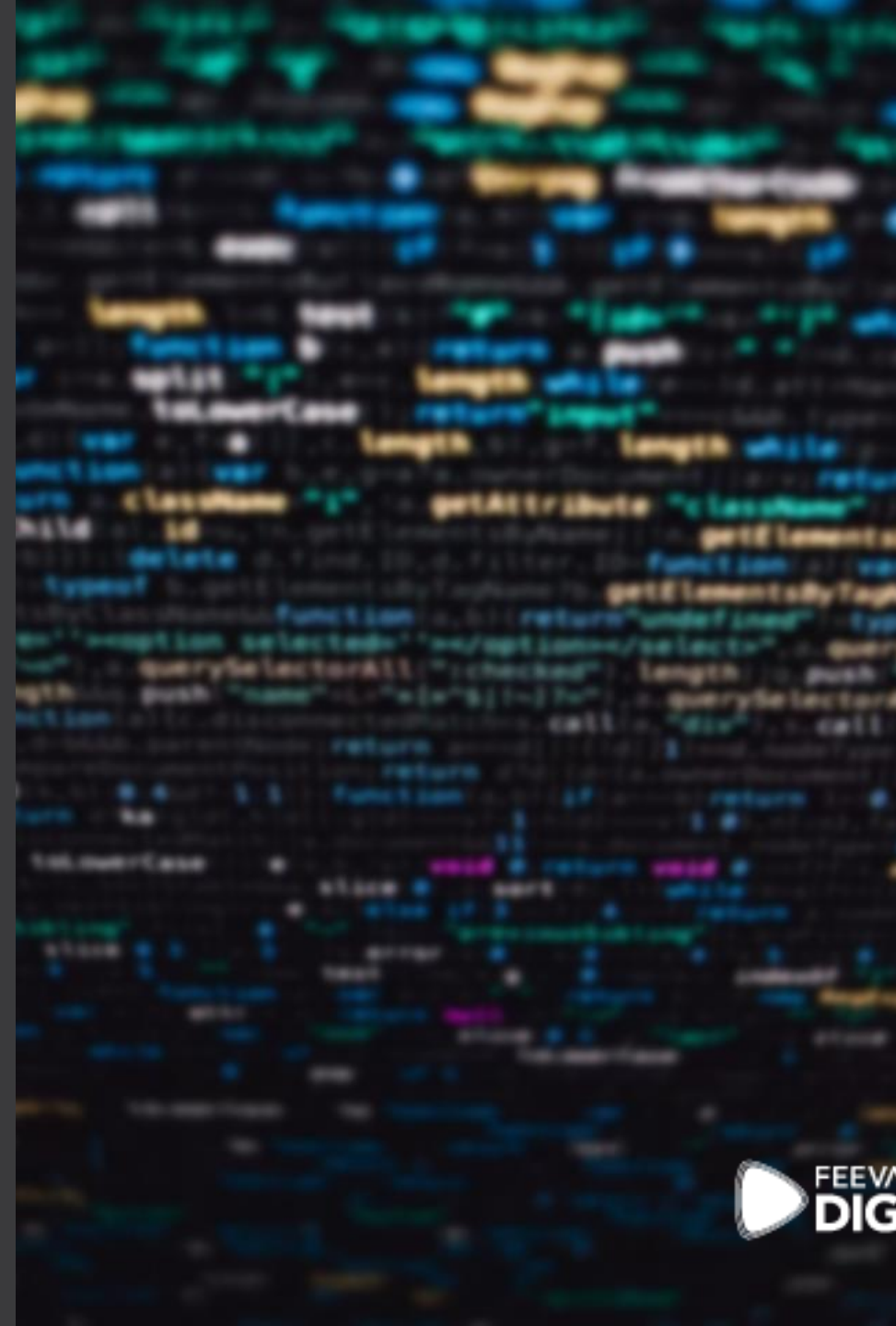


APRESENTAÇÃO



INTRODUÇÃO AO JAVA SWING

Prof. Dr. Ricardo Ferreira de Oliveira





JAVA SWING

O Java Swing é uma Interface de Usuário Gráfica (GUI) que é compatível com a interface anterior conhecida como AWT (*Abstract Window Toolkit*)

Ela dispõe de diversos componentes com funcionalidades variadas dos quais veremos alguns dos principais neste curso.

Nesta apresentação discorreremos sobre os componentes que serão vistos nos exemplos iniciais de modo que o aluno possa com facilidade acompanhar o desenvolvimento da aplicação DalaFlix.



JFRAME

Inicialmente apresentaremos o JFrame. O JFrame é uma “moldura” que é a base para inserirmos os diversos componentes de uma aplicação Java Swing.

Um JFrame pode aparecer na forma de uma caixa de diálogo (vistas anteriormente) ou uma janela.

Entre as ações que podem ser feitas com um JFrame estão as ações de minimizar, maximizar e fechar a janela.

No exemplo do slide a seguir, temos uma classe que herda da classe JFrame sem nada dentro de modo que temos as três opções mencionadas:



Esta aplicação cria um objeto JFrame que aparece sem nenhuma dimensão. Para criar o JFrame precisamos instanciar o objeto e torna-lo visível. Isto fizemos no método main:

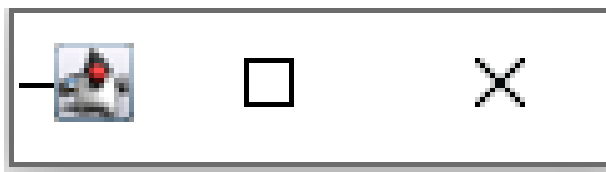
```
import javax.swing.JFrame;

public class JavaSwing1 extends JFrame {

    public static void main(String[] args) {
        JavaSwing1 minhaJanela = new
        JavaSwing1();
        minhaJanela.setVisible( true );
    }
}
```



Executando o código do slide anterior obtemos o seguinte resultado:



Podemos redimensionar a janela simplesmente arrastando as suas bordas.



Podemos aprimorar o nosso código determinando um tamanho inicial para a Janela. Usando o método `setSize`, inicializamos a nossa janela com largura de 300 píxeis e altura de 200 píxeis:

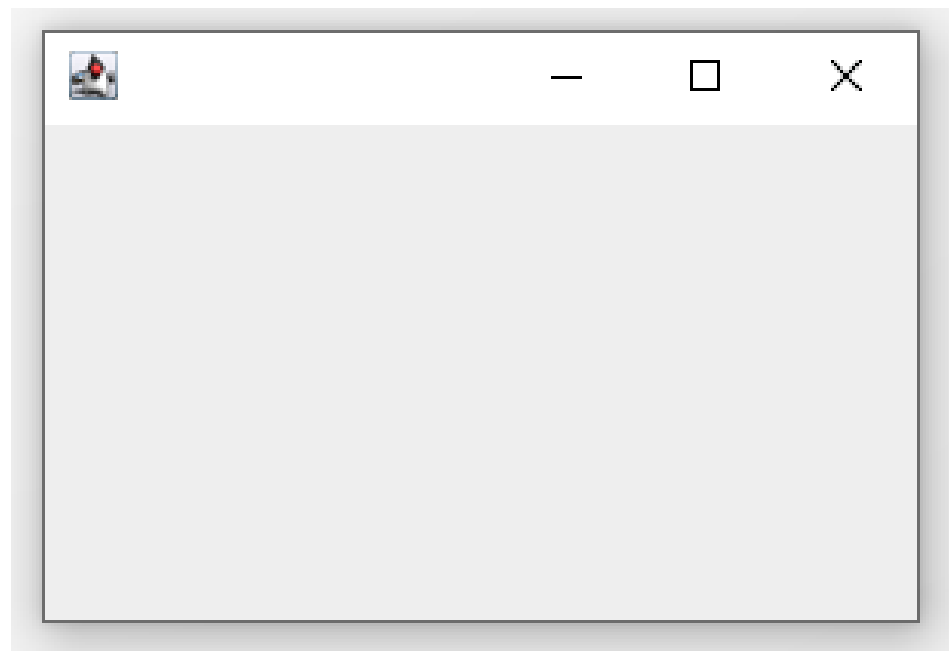
```
import javax.swing.JFrame;

public class JavaSwing2 extends JFrame {

    public static void main(String[] args) {
        JavaSwing2 minhaJanela = new
        JavaSwing2();
        minhaJanela.setSize(300,200);
        minhaJanela.setVisible( true );
    }
}
```



Executando o código do slide anterior obtemos o seguinte resultado:





Um aspecto importante quando lidamos com objetos JFrame é o que fazer com o objeto após ele ser fechado. Para tal podemos tomar as seguintes ações:

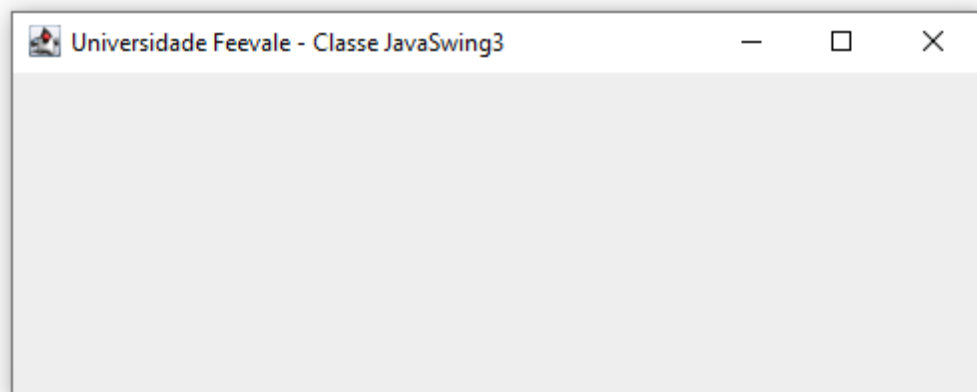
Ação	Descrição
<code>setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);</code>	Faz a aplicação sair mas deixa recursos para trás.
<code>setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);</code>	Faz a aplicação sair e disponibiliza a memória de volta ao sair.
<code>setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);</code>	Não faz nada ao fechar a janela.
<code>setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);</code>	Esconde a janela ao se clicar em fechar.



Aprimorando o nosso exemplo, determinamos um construtor e invocando o construtor da superclasse (“super”) demos um título a nossa janela. Usamos o JFrame.DISPOSE_ON_CLOSE ao fechar a janela.

```
import javax.swing.JFrame;
public class JavaSwing3 extends JFrame {
    public JavaSwing3() {
        super( "Universidade Feevale -
Classe JavaSwing3" );
    }
    public static void main(String[] args) {
        JavaSwing3 minhaJanela = new
JavaSwing3();
        minhaJanela.setSize(500,200);

        minhaJanela.setDefaultCloseOperation(
JFrame.DISPOSE_ON_CLOSE );
        minhaJanela.setVisible( true );
    }
}
```



Executando o código do slide anterior obtemos o seguinte resultado:



Aprimorando o nosso exemplo, determinamos um construtor e invocando o construtor da superclasse ("super") demos um título a nossa janela. Usamos o JFrame.DISPOSE_ON_CLOSE ao fechar a janela.

```
import javax.swing.JFrame;
public class JavaSwing4 extends JFrame {
    public JavaSwing4() {
        super( "Universidade Feevale -
Classe JavaSwing4" );
        setSize(500,200);
        setDefaultCloseOperation(
JFrame.DISPOSE_ON_CLOSE );
    }
    public static void main(String[] args) {
        JavaSwing4 minhaJanela = new
JavaSwing4();
        minhaJanela.setVisible( true );
    }
}
```



LAYOUTS

Podemos usar ao inserir outros componentes em nosso JFrame diversos tipos de Layouts através do método `setLayout`. Os mais comuns são os seguintes

LAYOUT	Descrição
null	Layout nulo, que possibilita a inserção de componentes em pontos fixos do JFrame.
FlowLayout	Este tipo de layout vai enfileirando os componentes em uma fileira e rearranja em mais de uma linha a medida que o espaço diminui.
BorderLayout	Divide o espaço do JFrame em cinco áreas: <code>BorderLayout.CENTER</code> , <code>BorderLayout.NORTH</code> , <code>BorderLayout.SOUTH</code> , <code>BorderLayout.EAST</code> , <code>BorderLayout.WEST</code>
GridLayout	Divide o espaço do JFrame em uma matriz de n linhas por m colunas especificadas quando na instanciação do GridLayout.

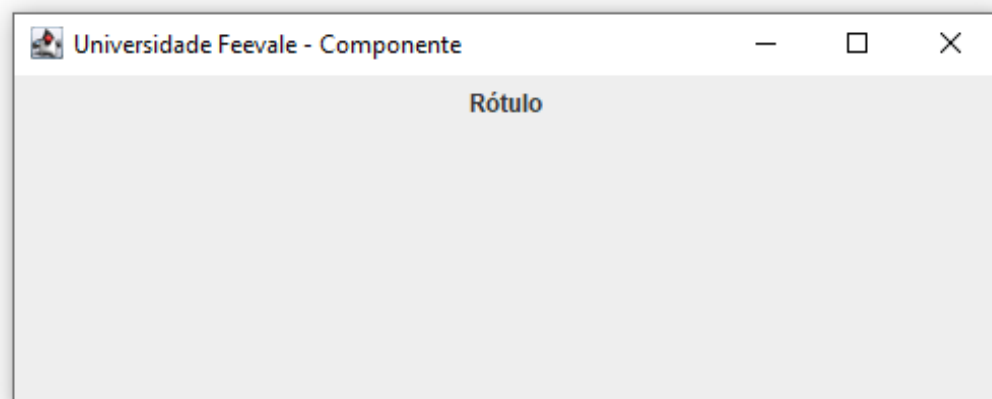
Para se aprofundar mais em layouts visite: <https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html>



JLABEL

O primeiro componente que iremos adicionar em nosso JFrame é o JLabel. O JLabel é um componente que serve para adicionar um rótulo ao nosso painel. Observe que precisamos inicializar o componente invocando o método construtor. Incluiremos o rótulo com layout FlowLayout:

```
1 import java.awt.FlowLayout;
2 import javax.swing.JFrame;
3 import javax.swing.JLabel;
4 public class JavaSwing5 extends JFrame {
5
6     private static final long serialVersionUID = 1L;
7     private JLabel rotulo;
8
9     public JavaSwing5() {
10         super( "Universidade Feevale - Componente" );
11
12         setLayout(new FlowLayout());
13         rotulo = new JLabel("Rótulo");
14         add(rotulo);
15
16         setSize(500,200);
17         setDefaultCloseOperation( JFrame.DISPOSE_ON_CLOSE );
18     }
19     public static void main(String[] args) {
20         JavaSwing5 minhaJanela = new JavaSwing5();
21         minhaJanela.setVisible( true );
22     }
23 }
```



Executando o código do slide anterior obtemos o seguinte resultado:



Agora iremos alterar um pouco o código anterior usando o layout nulo e determinando a posição exata de onde ficará o nosso rótulo. Para isso, usamos o método `setBounds` que tem 4 parâmetros:

- Deslocamento horizontal (coordenada X)
- Deslocamento vertical (coordenada Y)
- Largura
- Altura

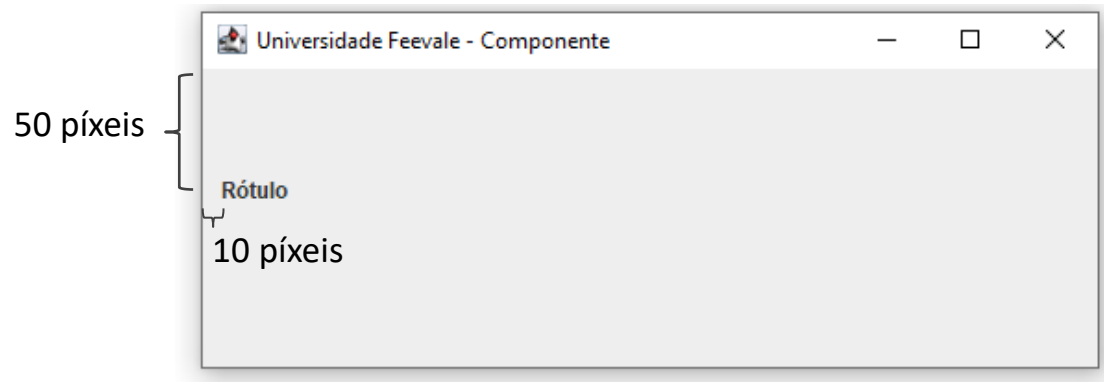
A parte do código alterado é essa:

```
setLayout(null);  
rotulo = new JLabel("Rótulo");  
rotulo.setBounds(10, 50, 200, 30 );  
add(rotulo);
```



O código completo é este:

```
1 import java.awt.FlowLayout;
4 public class JavaSwing6 extends JFrame {
5
6     private static final long serialVersionUID = 1L;
7     private JLabel rotulo;
8
9     public JavaSwing6() {
10         super( "Universidade Feevale - Componente" );
11
12         setLayout(null);
13         rotulo = new JLabel("Rótulo");
14         rotulo.setBounds(10, 50, 200, 30 );
15         add(rotulo);
16
17         setSize(500,200);
18         setDefaultCloseOperation( JFrame.DISPOSE_ON_CLOSE );
19     }
20     public static void main(String[] args) {
21         JavaSwing6 minhaJanela = new JavaSwing6();
22         minhaJanela.setVisible( true );
23     }
24 }
```

JLABEL

Executando o código do slide anterior obtemos o seguinte resultado:



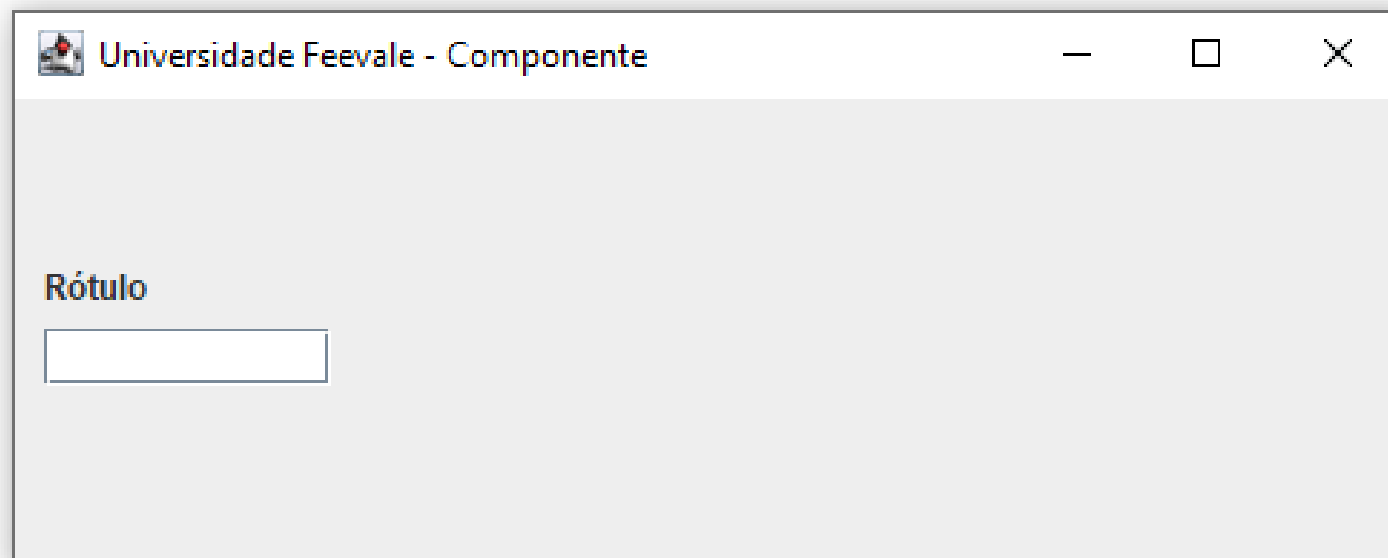
JTEXTFIELD

Nosso próximo componente é o componente JTextField. Este componente serve para a digitação de campos. O trecho de código alterado está abaixo:

```
9
10
11 private JTextField campoTexto;
12
13
14 public JavaSwing7() {
15     super( "Universidade Feevale - Componente" );
16
17     setLayout(null);
18     rotulo = new JLabel("Rótulo");
19     rotulo.setBounds(10, 50, 200, 30 );
20     add(rotulo);
21
22     campoTexto = new JTextField();
23     campoTexto.setBounds(10, 80, 100, 20 );
24     add(campoTexto);
25
26 }
```



Executando o código do slide anterior obtemos o seguinte resultado:



A rectangular button with a light blue gradient and a thin black border. The text "Aperte Aqui!" is centered on the button in a bold, black, sans-serif font. The button is set against a light gray background.

Aperte Aqui!

JBUTTON

Nosso próximo componente é o componente JButton. Este componente serve para criar botões. Botões necessitam que suas ações sejam “escutadas”. Em Java Swing isto se faz através de listeners. Inicialmente não nos preocuparemos com os listeners e incluiremos um botão em nosso JFrame.

O nosso programa então ficará como no slide seguinte:

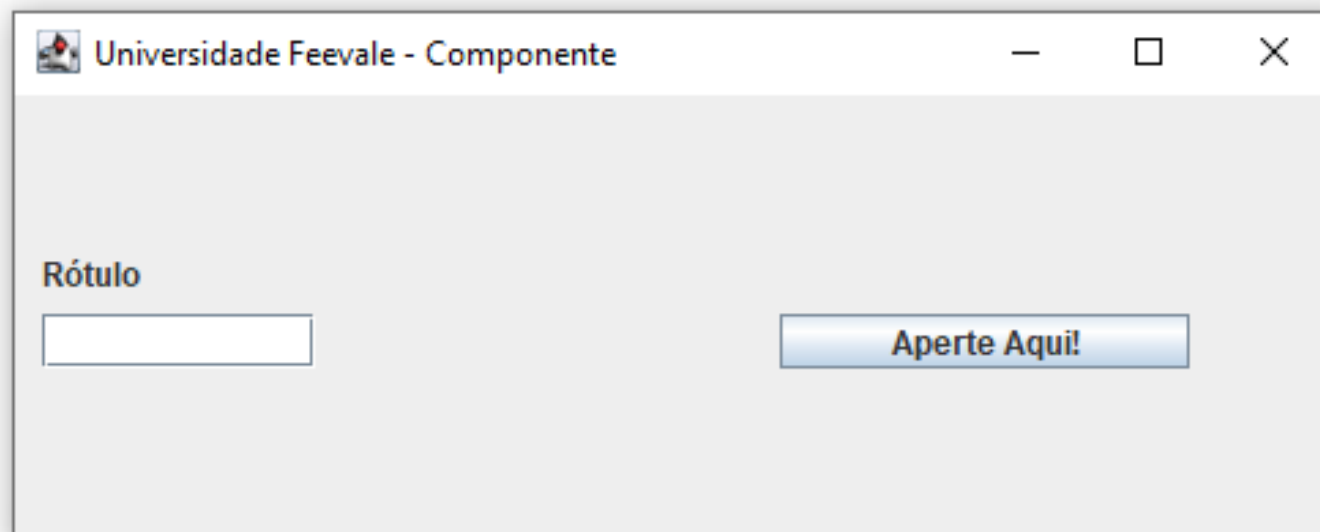


Trecho alterado:

```
10 private JLabel rotulo;  
11 private JTextField campoTexto;  
12 private JButton botao;  
13  
14 public JavaSwing8() {  
15     super( "Universidade Feevale - Componente" );  
16  
17     setLayout(null);  
18     rotulo = new JLabel("Rótulo");  
19     rotulo.setBounds(10, 50, 200, 30 );  
20     add(rotulo);  
21  
22     campoTexto = new JTextField();  
23     campoTexto.setBounds(10, 80, 100, 20 );  
24     add(campoTexto);  
25  
26     botao = new JButton( "Aperte Aqui!");  
27     botao.setBounds(280, 80, 150, 20 );  
28     add(botao);
```



Executando o código do slide anterior obtemos o resultado abaixo. Porém, o botão estará sem ação. Apertar no botão não resultará em nada.





LISTENERS

Então vamos agora conferir uma ação para o nosso botão. Para isso podemos usar um ActionListener. Podemos fazer isso de 2 formas: podemos fazer com que o nosso JFrame se torne um ActionListener ou podemos criar um objeto ActionListener dentro de nossa aplicação.

Utilizaremos a segunda forma aqui. Mais adiante, usaremos a primeira forma.

Existem outros tipos de listeners que iremos ver ao longo do curso.

No slide seguinte mostramos os passos para darmos a capacidade de nossa aplicação de responder a uma ação de pressionamento do botão.



CRIANDO UMA AÇÃO PARA NOSSO BOTÃO

De início instanciaremos um objeto que será a ação de nosso botão:

```
ActionListener acaoParaOBotao = new ActionListener(){  
    public void actionPerformed(ActionEvent arg0) {  
        if( arg0.getActionCommand().equals( "Apertar" ) ) {  
            JOptionPane.showMessageDialog( null, "Apertou o botão" );  
        }  
    }  
};
```

O método `actionPerformed` é obrigatório em um `ActionListener` e recebe um objeto `ActionEvent`. Este por sua vez recebe o comando submetido e se for a String “Apertar” o nosso botão exibirá uma mensagem.



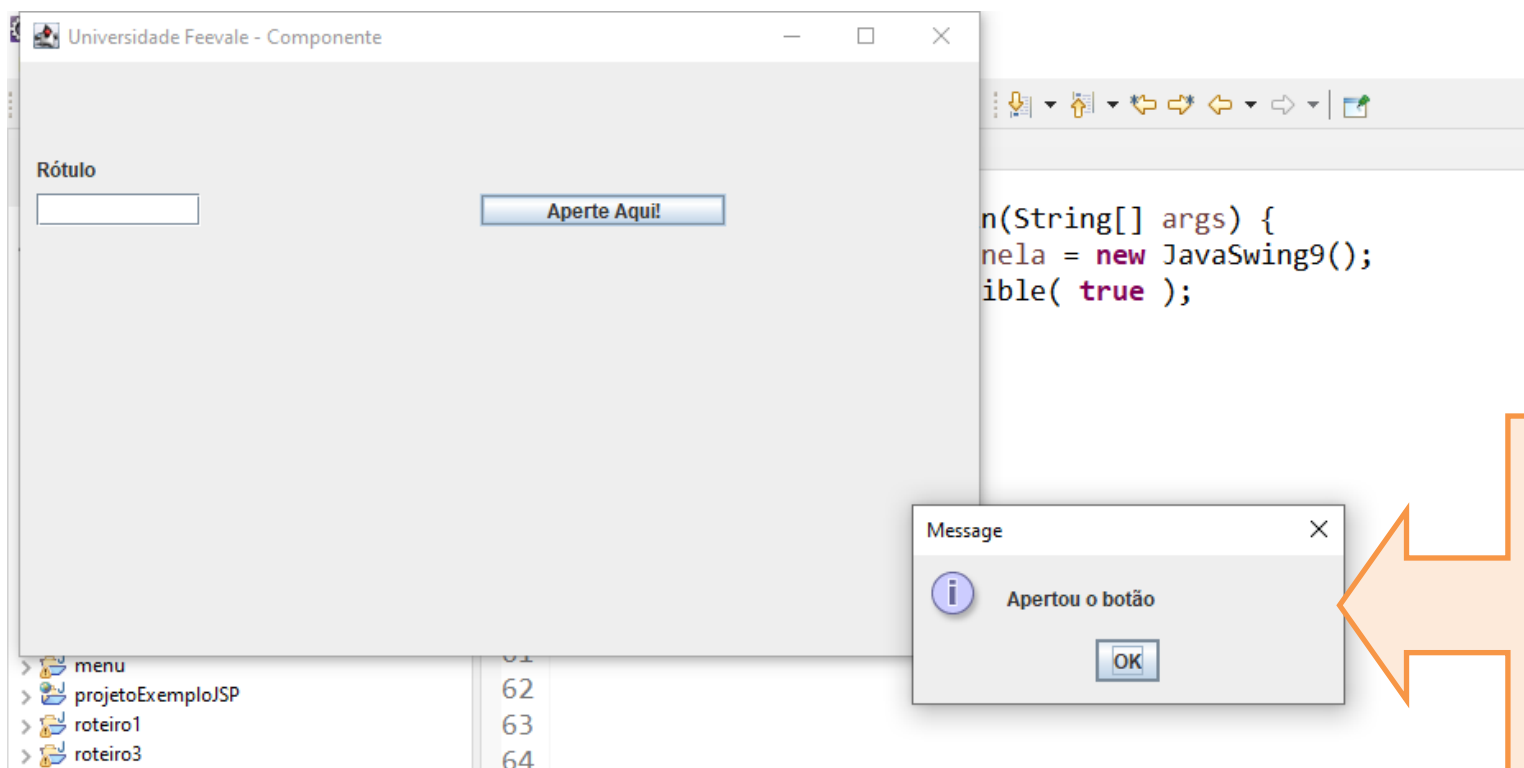
Iremos agora fazer duas alterações em nosso botão. Iremos estabelecer que ele será “escutado” pelo nosso ActionListener (acaoParaOBotao) e que, quando apertado, irá gerar uma ação “Apertar”.

```
botao = new JButton( "Aperte Aqui!");  
botao.setBounds(280, 80, 150, 20 );  
botao.setActionCommand( "Apertar" );  
botao.addActionListener( acaoParaOBotao );  
add(botao);
```

Desta forma, nosso botão ao ser apertado irá mostrar um aviso de que o botão foi apertado. No slide a seguir iremos mostrar a execução de nossa aplicação (JavaSwing9



JBUTTON



Observe que a mensagem ficou centralizada em relação a tela e não ficou dentro de nosso JFrame. Para que isso aconteça devemos no lugar de null, informar nosso componente JFrame



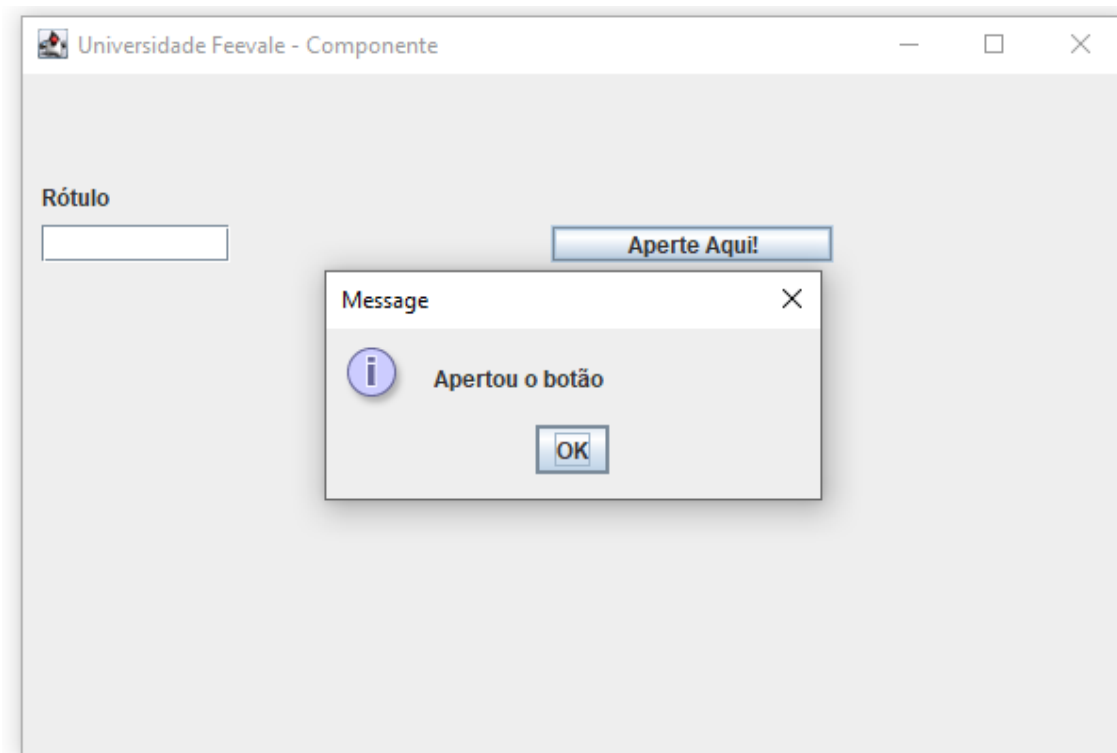
FAZENDO A MENSAGEM APARECER CENTRALIZADA NO JFRAME

Para que a mensagem apareça centralizada no JFrame, tudo que precisamos fazer é criar uma variável estática apontando para o JFrame e fornecer como o primeiro parâmetro do showMessageDialog.

```
ActionListener acaoParaOBotao = new ActionListener(){  
    public void actionPerformed(ActionEvent arg0) {  
        if( arg0.getActionCommand().equals( "Apertar" ) ) {  
            JOptionPane.showMessageDialog( minhaJanela, "Apertou o botão" );  
        }  
    }  
};
```



MENSAGEM CENTRALIZADA





MAIS COMPONENTES

Na próxima apresentação serão vistos mais exemplos de outros componentes e outros listeners.

É importante que o aluno teste cada um dos exemplos de modo a entender completamente o funcionamento de cada um deles



REFERÊNCIAS

DEITEL, Paul; Deitel, Harvey. Java: como programar. Editora Pearson 970 ISBN 9788543004792.

FURGERI, Sérgio. Java 7 ensino didático. 2. São Paulo Erica 2012 1 recurso online ISBN 9788536505251.